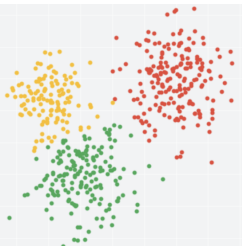


Clustering Problem

Procheta Sen



Clustering Problem

- ▶ Given a dataset, partition its objects into sets (**clusters**) $C_1, C_2, \dots, C_k$ such that the objects in each cluster are "**similar**" to one another.
- ▶ Specific definitions depend on how the notion of **similarity** is defined.

Why Cluster Data?

- Data summarization
- Topic detection
- Visualisation
- Outlier detection
- Community detection

Clustering is Unsupervised Learning

- Supervised learning
 - ▶ Labels for training instances are provided
- Unsupervised learning
 - ▶ No labels for training instances are provided
- Semi-supervised learning
 - ▶ Both labeled and unlabeled training instances are provided
- What can we learn about training data if we do not have any labels?
 - ▶ The similarity and distribution of the features can still be learned and this can be used to create rich feature spaces for supervised learning (if required)

How to cluster the following data

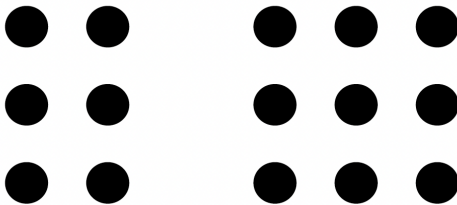


Figure: Clustering Demonstration

How to cluster the following data

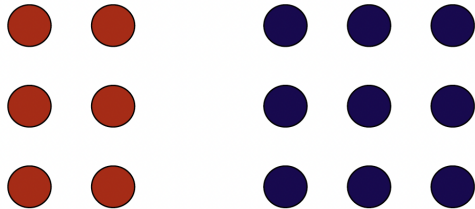


Figure: Clustering Demonstration

How to cluster the following data

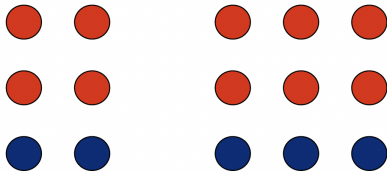
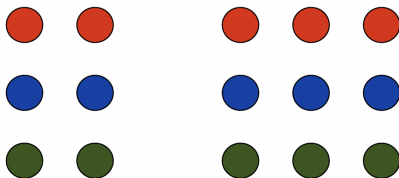


Figure: Clustering Demonstration

How to cluster the following data



How many clusters?

Figure: Clustering Demonstration

General Remarks

- A single dataset can be clustered in several ways
- There is no single right or wrong clustering
 - ▶ Simply different views on the same data
- Then how can we measure the quality of a clustering algorithm?
 - ▶ **Extrinsic methods:** Compare the clusters produced by a clustering algorithm against some reference (gold standard or ground truth) set of clusters
- **Intrinsic methods:** only the partition of objects into clusters is used

Clustering Algorithms

- **Representative-based**

- ▶ Choose k representatives, assign each element in the dataset to a representative, and iteratively update the partition
- ▶ k -Means, k -Medoids

- **Hierarchical**

- ▶ Create a hierarchy of clusters (dendrogram)
- ▶ Agglomerative clustering (bottom-up)
- ▶ Conglomerative clustering (top-down)

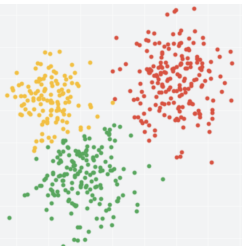
- **Graph-based clustering**

- ▶ Community detection (Modularity optimisation)
- ▶ Graph-cut algorithms (Spectral Clustering)

- **Many other types**

Clustering Evaluation

Procheta Sen



Clustering Evaluation

1. Extrinsic methods (supervised methods)

- ▶ ground truth labels are used
- ▶ assign a score to a clustering given the ground truth labels

2. Intrinsic methods (unsupervised methods)

- ▶ only the partition of the objects into clusters is used
- ▶ examine how well the clusters are separated and how compact the clusters are

Extrinsic methods: evaluate clustering as classification

1. Assign each cluster the label that appears most in that cluster
2. Merge clusters with the same label
3. Measure Precision, Recall, and F-measure for each label type
4. Compute the macro-average, i.e. the average over all label types (classes), of
 - ▶ Precision
 - ▶ Recall
 - ▶ F-score

Extrinsic methods: evaluate clustering as classification

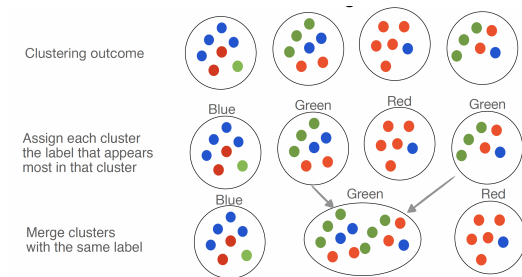


Figure:

Extrinsic methods: B-CUBED Measure

- ▶ Proposed in (Bagga B. Baldwin = B³)
 - ▶ A. Bagga and B. Baldwin. *Entity-based cross document coreference resolution using the vector space model*, In Proc. of 36th COLING-ACL, pages 79–85, 1998.
- ▶ We would like to evaluate clustering without labelling any clusters.

$$\text{precision}(x) = \frac{\text{No. of items in } C(x) \text{ with } A(x)}{\text{No. of items in } C(x)}$$

$$\text{recall}(x) = \frac{\text{No. of items in } C(x) \text{ with } A(x)}{\text{Total no. of items with } A(x)}$$

$A(x)$: label of x

$C(x)$: The ID of the cluster that x belongs to

Extrinsic methods: B-CUBED Measure

- Compute the average over all the items (instances) that appear in all clusters (N)

$$\text{Precision} = \frac{1}{N} \sum_{p \in \text{DataSet}} \text{Precision}(p)$$

$$\text{Recall} = \frac{1}{N} \sum_{p \in \text{DataSet}} \text{Recall}(p)$$

$$F\text{-Score} = \frac{1}{N} \sum_{p \in \text{DataSet}} F(p)$$

Intrinsic methods: Silhouette Coefficient

1. Let $C_1, C_2, \dots, C_k$ be the clusters
2. For object x (assume $x \in C_i$):
 - ▶ $a(x)$: The mean distance between x and all other points in the cluster of x .
 - ▶ $b(x)$: The mean distance between x and all other points in the *next nearest cluster*.

$$a(x) = \frac{1}{|C_i| - 1} \sum_{\substack{y \in C_i \\ y \neq x}} d(x, y)$$

$$b(x) = \min_{\substack{j=1, \dots, k \\ j \neq i}} \frac{1}{|C_j|} \sum_{y \in C_j} d(x, y)$$

$a(x)$ is a measure of how dissimilar x is to its own cluster, a small value means it is well matched.

$b(x)$ is a measure of how badly x is matched to its neighbouring cluster.

Intrinsic methods: Silhouette Coefficient

3. Silhouette Coefficient of x

$$s(x) = \frac{b(x) - a(x)}{\max\{a(x), b(x)\}}, \quad \text{if } |C_i| > 1$$

$$s(x) = 0, \quad \text{if } |C_i| = 1$$

$s(x)$ close to 1 means that the data is appropriately clustered

$s(x)$ close to -1 means that it would be more appropriate if x was clustered in its neighbouring cluster

$s(x)$ close to 0 means that x is on the border of two natural clusters

4. Silhouette Coefficient of the dataset is the average of Silhouette Coefficients of all objects in the dataset

measure of how appropriately the data have been clustered

Intrinsic methods: Silhouette Coefficient

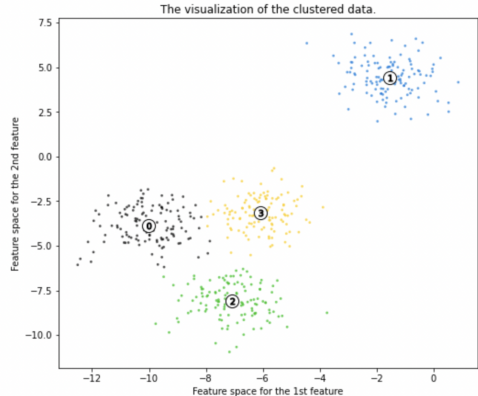
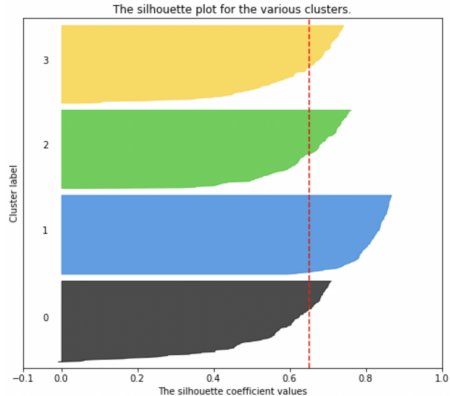
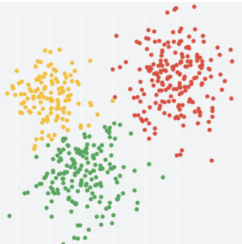


Figure:

K-Means Algorithm

Procheta Sen



Representative-based algorithms

- Let k be the number of clusters
- Let $D = \{\bar{X}_1, \dots, \bar{X}_n\}$ be the dataset
- The goal is to determine k representatives $\bar{Y}_1, \dots, \bar{Y}_k$ that minimise the following objective function

$$\sum_{i=1}^n \left[\min_{1 \leq j \leq k} d(\bar{X}_i, \bar{Y}_j) \right]$$

i.e. the sum of the distances of the objects to their closest representatives needs to be minimised.

Representative-based algorithms

We obtain specific algorithms by specifying

- the way of choosing representatives, and
- the distance function $d(\cdot, \cdot)$

In general representatives do not necessarily belong to the dataset.

General k -representatives approach

- **Initialise:** pick initial k representatives
- **Iteratively refine:**
 - ▶ **(assign step)** Assign each object to its closest representative using distance function $d(\cdot, \cdot)$. Denote the corresponding clusters $C_1, \dots, C_k$.
 - ▶ **(optimise step)** Determine the optimal representative $\bar{Y}_j$ for each cluster C_j that minimises its local objective function

$$\sum_{\bar{X}_i \in C_j} d(\bar{X}_i, \bar{Y}_j)$$

k-Means algorithm

- ▶ Representatives are chosen **not necessarily** from the dataset
- ▶ The distance function is **squared Euclidean distance**

k -Means algorithm

The objective function

$$\min_{\bar{Y}_1, \dots, \bar{Y}_k} \sum_{i=1}^k \sum_{\bar{X} \in C_i} \|\bar{X} - \bar{Y}_i\|^2$$

where C_i consists of the objects that are closest to $\bar{Y}_i$.

We want to minimise the total **squared Euclidean distance** between data objects and their cluster representatives $\bar{Y}_1, \dots, \bar{Y}_k$

This objective function is called the *within cluster sum of squares (WCSS)* objective

k -Means algorithm

Assume that the clusters $C_1, C_2, \dots, C_k$ are fixed.

Find the set $\bar{Y}_1, \bar{Y}_2, \dots, \bar{Y}_k$ of representatives such that

$$f_{C_1, \dots, C_k}(\bar{Y}_1, \dots, \bar{Y}_k) = \sum_{i=1}^k \sum_{\bar{X} \in C_i} \|\bar{X} - \bar{Y}_i\|^2$$

is minimised.

$$\frac{\partial f_{C_1, \dots, C_k}(\bar{Y}_1, \dots, \bar{Y}_k)}{\partial \bar{Y}_i} = - \sum_{\bar{X} \in C_i} 2(\bar{X} - \bar{Y}_i) = 0$$
$$\bar{Y}_i = \frac{1}{|C_i|} \sum_{\bar{X} \in C_i} \bar{X}$$

Just compute the centroid (mean) of each cluster and that will give you the new **optimal** cluster representatives

KMeans Illustration

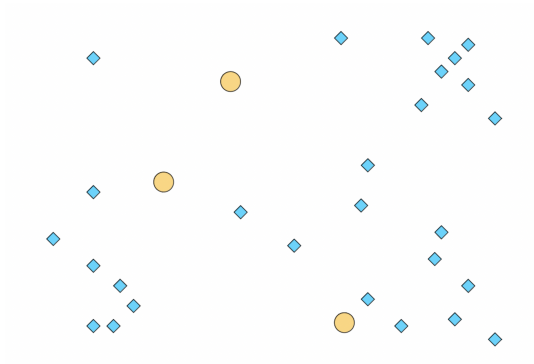


Figure: K Means Demonstration

KMeans Illustration

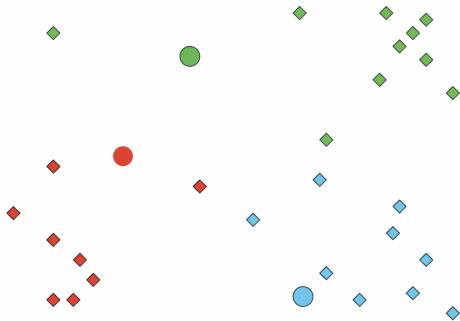


Figure: K Means Demonstration

KMeans Illustration

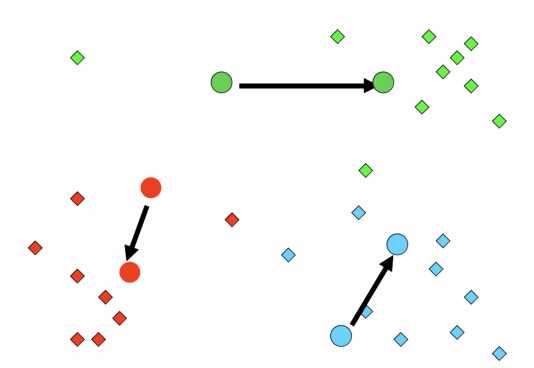


Figure: K Means Demonstration

KMeans Illustration

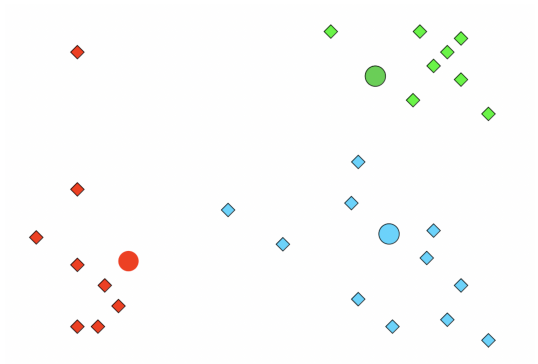


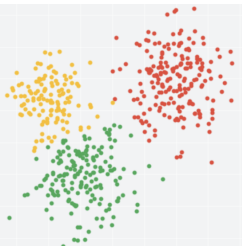
Figure: K Means Demonstration

Issues with k -Means algorithm

- ▶ Results can vary depending on initial random choices
- ▶ Can get trapped in a local minimum that isn't the global optimal solution
- ▶ Repeat the clustering procedure multiple times with different initialisations and select the *best* final clustering
- ▶ Outliers have a larger effect on the mean value, hence cluster centre and the cluster
- ▶ Cluster centres (means) are not actual instances in the cluster
- ▶ Euclidean distance used in the algorithm is inappropriate for categorical features

How to choose initial cluster representatives?

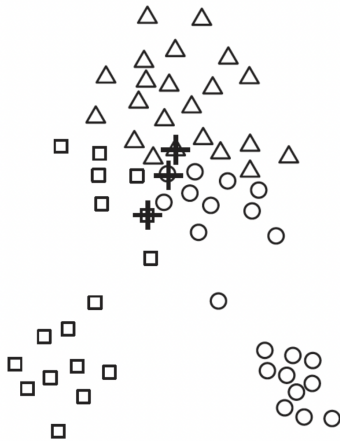
Procheta Sen



How to choose initial cluster representatives

1. Randomly
2. Randomly repeat several times
3. Sampling + hierarchical clustering
4. Furthest points
5. k -means++

Choosing randomly: lucky choice



Iteration 1

Choosing randomly: lucky choice

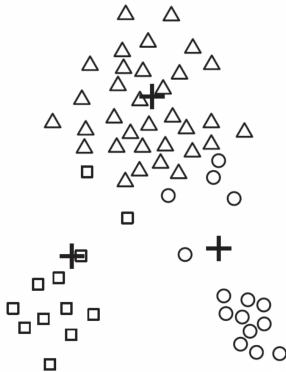


Figure: Iteration 2

Choosing randomly: lucky choice

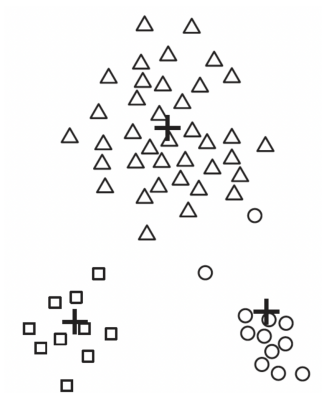


Figure: Iteration 3

Choosing randomly: lucky choice

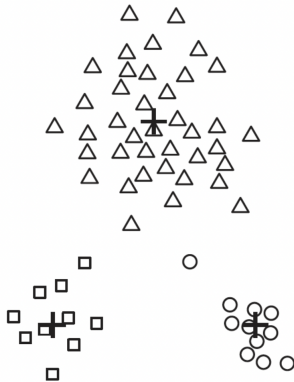


Figure: Iteration 4

Choosing randomly: unlucky choice

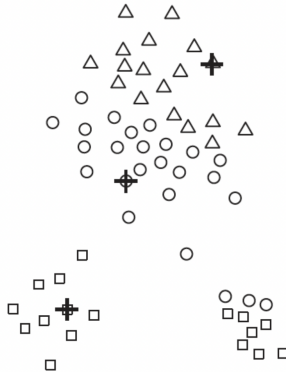


Figure: Iteration 1

Choosing randomly: unlucky choice

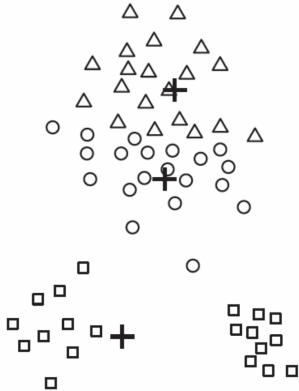


Figure: Iteration 2

Choosing randomly: unlucky choice

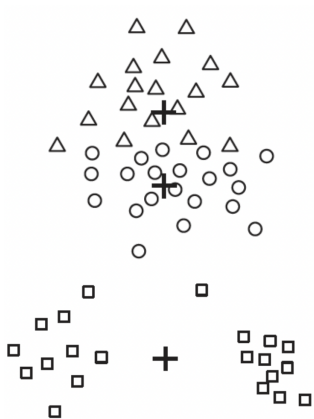


Figure: Iteration 3

Choosing randomly: unlucky choice

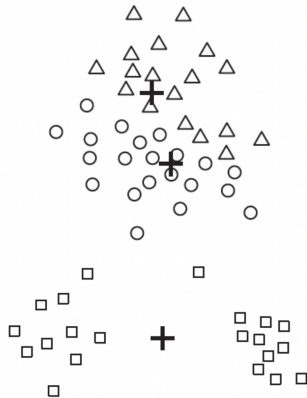
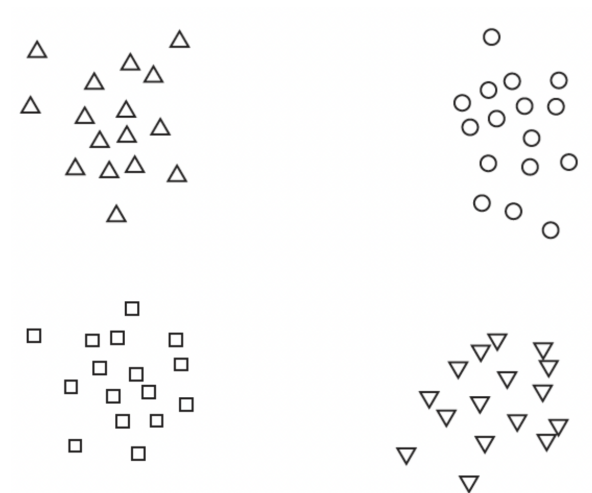


Figure: Iteration 4

Choosing randomly several times

1. Select initial cluster representatives randomly
2. Run k-means
3. Repeat steps 1–2 several times and choose the best clustering

Choosing randomly several times: lucky choice



Choosing randomly several times: lucky choice

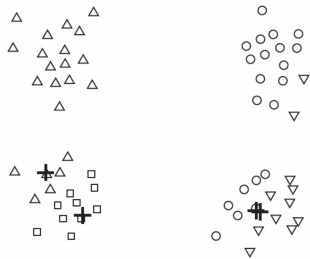


Figure: Iteration 1

Choosing randomly several times: lucky choice

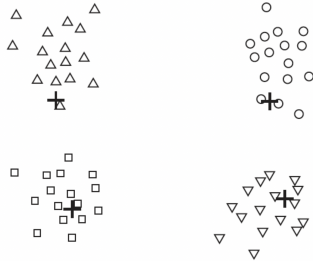


Figure: Iteration 2

Choosing randomly several times: lucky choice

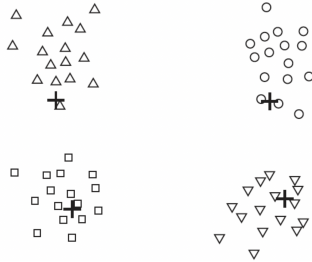


Figure: Iteration 3

Choosing randomly several times: unlucky choice

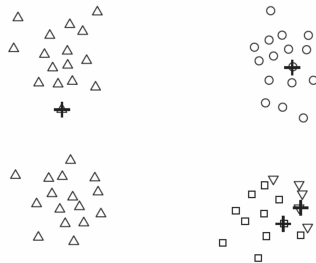


Figure: Iteration 1

Choosing randomly several times: unlucky choice

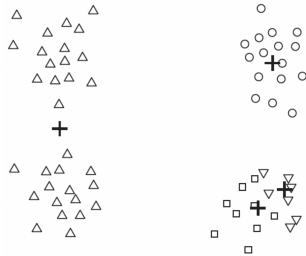


Figure: Iteration 2

Choosing randomly several times: unlucky choice

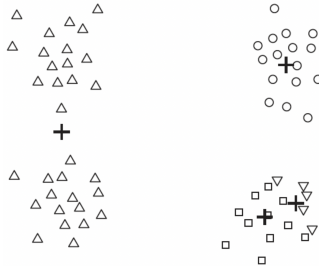


Figure: Iteration 3

Choosing randomly several times: unlucky choice

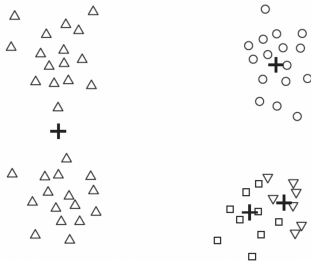


Figure: Iteration 4

Sampling + hierarchical clustering

1. Sample subset $\mathcal{D}'$ of points from the dataset $\mathcal{D}$
2. Cluster $\mathcal{D}'$ using a hierarchical clustering technique.
3. Extract k clusters from the hierarchical clustering.
4. Compute the means of these k clusters, and use them as the initial cluster representatives
5. Proceed with the standard k -means with these cluster representatives

Sampling + hierarchical clustering

Often works well, but it is practical only if

1. The sampled subset $\mathcal{D}'$ is relatively small (a few hundred to a few thousand), as hierarchical clustering is expensive
2. k is relatively small compared to the size of the sampled set $\mathcal{D}'$

Selecting furthest points

For an object $\bar{X}$ in the dataset $\mathcal{D}$ let $R(\bar{X})$ be the distance from $\bar{X}$ to the closest cluster representative we have already chosen.

1. Select one representative $\bar{Y}_1$ uniformly at random from $\mathcal{D}$.
2. For every $i = 2, \dots, k$
 - 2.1 Select representative $\bar{Y}_i$ from $\mathcal{D}$ with the maximum value of $R(\cdot)$
3. Proceed with the standard k-means using $\bar{Y}_1, \dots, \bar{Y}_k$ as initial cluster representatives

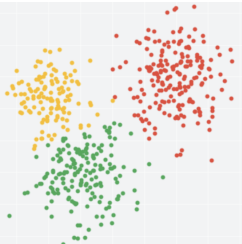
Selecting furthest points

The main drawback:

such an approach can select outliers, rather than points in clusters

KMeans++

Procheta Sen



k-means++

For an object $\bar{X}$ in the dataset $\mathcal{D}$ let $R(\bar{X})$ be the distance from $\bar{X}$ to the closest cluster representative we have already chosen.

1. Select one representative $\bar{Y}_1$ uniformly at random from $\mathcal{D}$.
2. For every $i = 2, \dots, k$
 - 2.1 Select representative $\bar{Y}_i$ from $\mathcal{D}$ with probability

$$\Pr(\bar{Y}_i = \bar{X}) = \frac{R(\bar{X})^2}{\sum_{\bar{X} \in \mathcal{D}} R(\bar{X})^2}$$

3. Proceed with the standard k -means using $\bar{Y}_1, \dots, \bar{Y}_k$ as initial cluster representatives

k-means++

Within cluster sum of squares:

$$\phi = \sum_{x \in \mathcal{X}} \min_{c \in \mathcal{C}} \|x - c\|^2.$$

Theorem 3.1. *If $\mathcal{C}$ is constructed with k-means++, then the corresponding potential function ϕ satisfies*

$$\mathbb{E}[\phi] \leq 8(\ln k + 2)\phi_{\text{OPT}}.$$

k-means++ vs k-means: experimental study

- ▶ Synthetic datasets Norm-10 and Norm-25.
 - ▶ Chose 10 (respectively 25) “real” centers uniformly at random from the hypercube of side length 500.
 - ▶ Then added points from a Gaussian distribution of variance 1, centered at each of the real centers.
 - ▶ Thus, we have a number of well separated Gaussians with the real centers providing a good approximation to the optimal clustering.

k-means++ vs k-means: experimental study

- Synthetic dataset Norm-10.

k	Average ϕ		Minimum ϕ	
	k-means	k-means++	k-means	k-means++
10	10898	5.122	2526.9	5.122
25	787.992	4.46809	4.40205	4.41158
50	3.47662	3.35897	3.40053	3.26072

Table: Experimental results on the Norm-10 dataset ($n = 10000$, $d = 5$)

Within cluster sum of squares:
$$\phi = \sum_{x \in \mathcal{X}} \min_{c \in \mathcal{C}} \|x - c\|^2$$

k-means++ vs k-means: experimental study

- Synthetic dataset Norm-25.

k	Average ϕ		Minimum ϕ	
	k-means	k-means++	k-means	k-means++
10	135512	126433	119201	111611
25	48050.5	15.8313	25734.6	15.8313
50	5466.02	14.76	14.79	14.73

Table: Experimental results on the Norm-25 dataset ($n = 10000$, $d = 15$)

Within cluster sum of squares:
$$\phi = \sum_{x \in \mathcal{X}} \min_{c \in \mathcal{C}} \|x - c\|^2$$

k-means++ vs k-means: experimental study

- ▶ The synthetic datasets: the k-means method does not perform well, because
 - ▶ the random seeding will inevitably merge clusters together, and the algorithm will never be able to split them apart.
 - ▶ the careful seeding method of k-means++ avoids this problem altogether, and it almost always attains the optimal results on the synthetic datasets

k-means++ vs k-means: experimental study

- Real datasets: Cloud dataset

k	Average ϕ		Minimum ϕ	
	k-means	k-means++	k-means	k-means++
10	7553.5	6151.2	6139.45	5631.99
25	3626.1	2064.9	2568.2	1988.76
50	2004.2	1133.7	1344	1088

Table: Experimental results on the *Cloud* dataset ($n = 1024, d = 10$)

Within cluster sum of squares:

$$\phi = \sum_{x \in X} \min_{c \in C} \|x - c\|^2.$$

k-means++ vs k-means: experimental study

► Real datasets: Intrusion dataset

k	Average ϕ		Minimum ϕ	
	k-means	k-means++	k-means	k-means++
10	$3.45 \cdot 10^8$	$2.31 \cdot 10^7$	$3.25 \cdot 10^8$	$1.79 \cdot 10^7$
25	$3.15 \cdot 10^8$	$2.53 \cdot 10^6$	$3.10 \cdot 10^8$	$2.06 \cdot 10^6$
50	$3.08 \cdot 10^8$	$4.67 \cdot 10^5$	$3.08 \cdot 10^8$	$3.98 \cdot 10^5$

Table: Experimental results on the *Intrusion* dataset ($n = 494019, d = 35$)

Within cluster sum of squares:

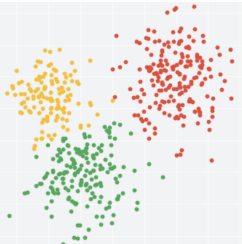
$$\phi = \sum_{x \in X} \min_{c \in C} \|x - c\|^2.$$

k-means++ vs k-means: experimental study

- ▶ The real-world datasets
- ▶ On the **Cloud** dataset, k-means++ terminates almost **twice as fast** while achieving potential function (within cluster sum of squares) values about **20% better**.
- ▶ On the larger **Intrusion** dataset: the potential value obtained by k-means++ is better by factors of **10 to 1000** and is also obtained **up to 70% faster**.

KMedoids

Procheta Sen



Issues with k -Means algorithm

- ▶ Results can vary depending on initial random choices
 - ▶ Can get trapped in a local minimum that isn't the global optimal solution
 - ▶ Repeat the clustering procedure multiple times with different initialisations and select the *best* final clustering
-
- ▶ Outliers have a larger effect on the mean value, hence cluster centre and the cluster
 - ▶ Cluster centres (means) are not actual instances in the cluster
 - ▶ Euclidean distance used in the algorithm is inappropriate for categorical features

k -Medoids algorithm

- ▶ Representative-based algorithms
- ▶ The goal is to determine k representatives $\bar{Y}_1, \dots, \bar{Y}_k$ that minimise the following objective function

$$\sum_{i=1}^n \left[\min_j d(\bar{X}_i, \bar{Y}_j) \right]$$

Unlike the k -Means, in the k -Medoids algorithm

- ▶ the **distance function** (dissimilarity measure) $d(\cdot, \cdot)$ can be any function convenient for the dataset (not necessarily the Euclidean distance)
- ▶ representatives are selected from the dataset

k -Medoids algorithm

Uses hill-climbing strategy:

1. Start with an arbitrary solution to a problem
2. Attempt to find a better solution by making an incremental change to the solution.
3. If the change produces a better solution, another incremental change is made to the new solution, and so on
4. Until no further improvements can be found.

k -Medoids algorithm

k -MedoidsClustering (Number of clusters: k , Dataset: $\mathcal{D} = \{\bar{X}_1, \dots, \bar{X}_n\}$)

1. Initialisation phase

Choose k cluster representatives (medoids) $\bar{Y}_1, \dots, \bar{Y}_k$ from the dataset randomly

2. Assignment phase

Assign all objects in the dataset to the closest representative.

The resulting clusters: $C_1, \dots, C_k$

3. Optimisation phase (hill-climbing step)

1. Find a pair $(\bar{X}, \bar{Y})$, where $\bar{X} \in \mathcal{D}$ and $\bar{Y} \in \{\bar{Y}_1, \dots, \bar{Y}_k\}$ such that
2. Replacing $\bar{Y}$ with $\bar{X}$ in the set of representatives leads to the greatest possible improvement in the objective function
3. If improvement is positive then replace $\bar{Y}$ with $\bar{X}$ and go to phase 2. Otherwise return current clusters $C_1, \dots, C_k$

k -Medoids algorithm

Pros

- ▶ Representatives are chosen from the dataset
 - ▶ allows for greater interpretability of the cluster representatives
 - ▶ more robust to noise and outliers than k -means
- ▶ Can be used with arbitrary dissimilarity measures
 - ▶ thus applicable to data of complex data type (categorical, mixed, time-series, etc.)

Cons

- ▶ Results can vary depending on initial random choices
- ▶ Can get trapped in a local minimum that isn't the global optimal solution
 - ▶ Repeat the clustering procedure multiple times with different initialisations and select the best final clustering
- ▶ Slower than the k -means algorithm

k -Medoids algorithm: time-complexity issue

If we have n objects in the dataset, then at each execution of the optimisation phase we need to compute $k \cdot n$ times the incremental objective-function change (this is too expensive).

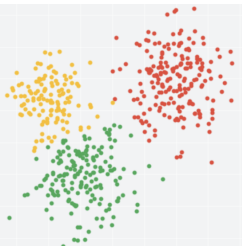
3. Optimisation phase (hill-climbing step)

1. Find a pair $(\bar{X}, \bar{Y})$, where $\bar{X} \in \mathcal{D}$ and $\bar{Y} \in \{\bar{Y}_1, \dots, \bar{Y}_k\}$, such that
2. Replacing $\bar{Y}$ with $\bar{X}$ in the set of representatives leads to the greatest possible improvement in the objective function.
3. If the improvement is positive then replace $\bar{Y}$ with $\bar{X}$ and go to phase 2. Otherwise return the current clusters $C_1, \dots, C_k$.

A solution to this is to use a randomly selected set of r pairs $(\bar{X}, \bar{Y})$, where $\bar{X} \in \mathcal{D}$ and $\bar{Y} \in \{\bar{Y}_1, \dots, \bar{Y}_k\}$, and use the best of these pairs for the replacement. In this way we need to compute only r times the incremental objective-function change.

Hierarchical Clustering

Procheta Sen



Why Hierarchical Clustering?

- ▶ No need to specify the number of clusters k
- ▶ Specifies clusterings at all granularities, simultaneously
- ▶ Can be used to organise the data into a hierarchy of subsuming concepts for visualisation (abstraction) purposes

Representation of Hierarchical Clustering

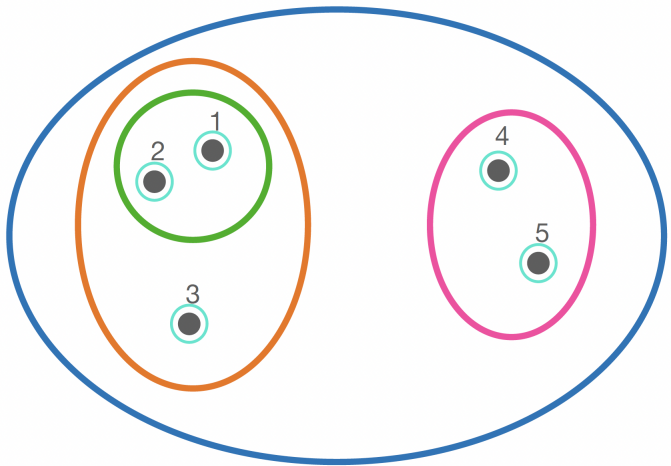
Hierarchy of clusters:

$\{1,2,3,4,5\}$

$\{1,2,3\}$ $\{4,5\}$

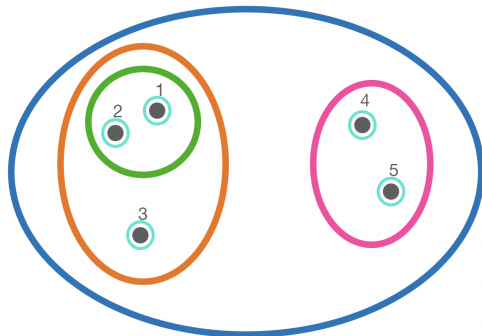
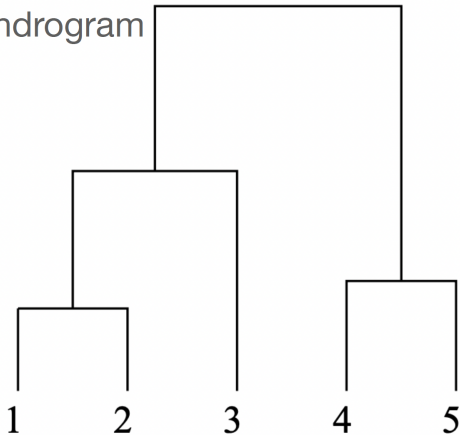
$\{1,2\}$ $\{3\}$ $\{4,5\}$

$\{1\}\{2\}\{3\}\{4\}\{5\}$



Representation of Hierarchical Clustering

Dendrogram



Hierarchy of clusters:

$\{1, 2, 3, 4, 5\}$

$\{1, 2, 3\} \{4, 5\}$

$\{1, 2\} \{3\} \{4, 5\}$

$\{1\} \{2\} \{3\} \{4\} \{5\}$

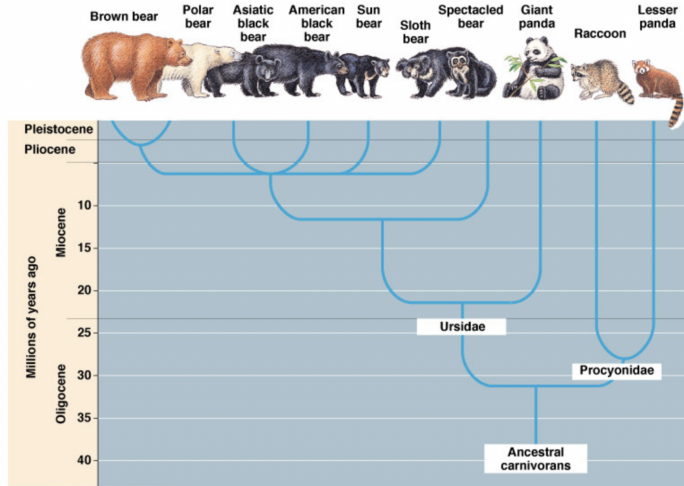
Example: Charting Evolution through Phylogenetic Trees

1. Generate the DNA sequences
2. Calculate the edit distance between all sequences.
3. Calculate the DNA similarities based on the edit distances.
4. Construct the phylogenetic tree.

`https:`

`//towardsdatascience.com/hierarchical-clustering-and-its-applications-41c1ad4441a6`

Example: Charting Evolution through Phylogenetic Trees



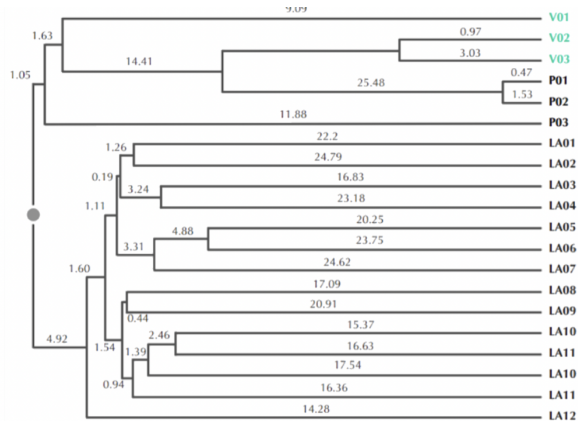
Example: Tracking Viruses through Phylogenetic Trees

- ▶ Viruses such as HIV have high mutation rates, which means the similarity of the DNA sequence of the same virus depends on the time since it was transmitted. This can be used to trace paths of transmission.
- ▶ Such method was used as evidence in a court case, wherein the victim's strand of HIV was found to be more similar to the accused patient's strand, compared to a control group.

`https:`

`//towardsdatascience.com/hierarchical-clustering-and-its-applications-41c1ad4441a6`

Example: Tracking Viruses through Phylogenetic Trees



V1-3 are victim's strands, P1-3 are accused patient's, and LA1-12 are the control group

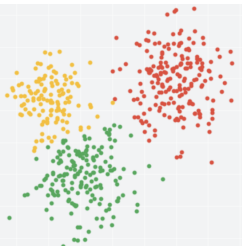
Hierarchical Clustering

Two approaches

- ▶ **Agglomerative** clustering
 - ▶ Start with singletons (clusters with exactly one instance) and iteratively merge the most *similar* two clusters
 - ▶ Bottom-up approach
- ▶ **Divisive** (conglomerative) clustering
 - ▶ Start from one big cluster with all data instances and repeatedly partition it
 - ▶ Top-down approach

Divisive Clustering

Procheta Sen



Divisive (top-down) methods: main idea

- ▶ A top-down approach is used to successively partition the data objects into a tree-like structure.
- ▶ A flat clustering algorithm $\mathcal{A}$ (e.g. k-means algorithm) may be used for the partitioning in a given step. The algorithm $\mathcal{A}$ can be any arbitrary clustering algorithm, not necessarily a distance based algorithm.

Divisive (top-down) methods: main idea

- ▶ Provide flexibility in terms of choosing the trade-off between the balance in the tree structure and the balance in the number of objects in each cluster. For example:
- ▶ **Strategy 1:** split the heaviest node (a cluster with the maximum number of objects). Will result in leaf nodes (clusters) with a similar number of objects in them.
- ▶ **Strategy 2:** split each cluster into the same number of subclusters. Will result in a balanced tree structure with the same number of children at each node, but the leaf nodes (most granular clusters) will have varying numbers of objects.

Generic divisive clustering algorithm

Input: dataset: $\mathcal{D}$; flat algorithm: $\mathcal{A}$

1. **Initialise** tree $\mathcal{T}$ to contain a single (root) vertex with entire dataset $\mathcal{D}$
2. **Repeat:**
 - 2.1 Select a leaf node L in $\mathcal{T}$ based on pre-defined criterion;
 - 2.2 Use algorithm $\mathcal{A}$ split L into $L_1, \dots, L_k$;
 - 2.3 Add $L_1, \dots, L_k$ as children of L in $\mathcal{T}$;
3. **Until** termination criterion
4. **Return** current clustering or hierarchy or clusterings

Bisecting k -Means algorithm

Input: dataset: $\mathcal{D}$; number of clusters: s

1. **Initialise** tree $\mathcal{T}$ to contain a single (root) vertex with entire dataset $\mathcal{D}$

2. **Repeat:**

2.1 Select a leaf node (cluster) L in $\mathcal{T}$ that has the largest sum of square distance

$$\sum_{\bar{X}, \bar{Y} \in L} \text{dist}(\bar{X}, \bar{Y})^2;$$

2.2 Split L into 2 clusters L_1, L_2 using k -means algorithm;

2.3 Add L_1, L_2 as children of L in $\mathcal{T}$;

3. **Until** the number of leaf clusters is s

4. **Return** the leaf clusters

Bisecting k-Means algorithm: example

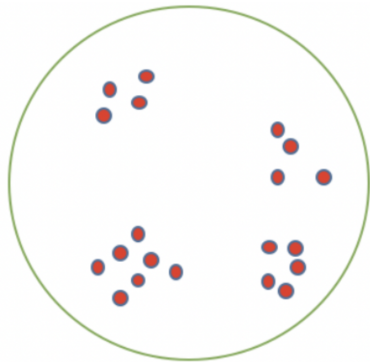


Figure:

Bisecting k-Means algorithm: example

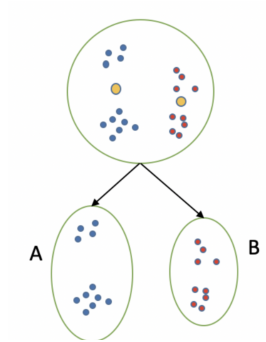


Figure:

Bisecting k-Means algorithm: example

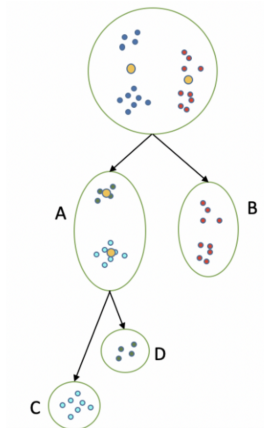


Figure:

Bisecting k-Means algorithm: example

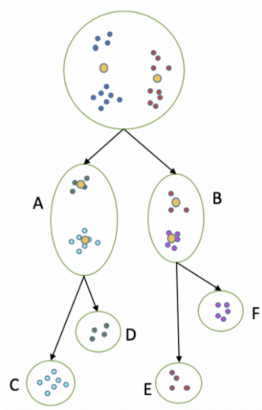


Figure: